

PROJETO DE BANCO DE DADOS
CALCULO DE SALÁRIOS

DCO3035 - UFRN

JOÃO RAFAEL DE SOUZA DANTAS
Professor: Hertz Wilton De Castro Lins

Prefácio

Este documento apresenta o desenvolvimento de um sistema web voltado para o gerenciamento de informações funcionais e o cálculo automatizado de salários em uma estrutura organizacional. Criado no contexto da disciplina DCO3035 - Banco de Dados da Universidade Federal do Rio Grande do Norte (UFRN), o projeto tem como base a tecnologia ASP.NET Web Forms, com banco de dados MySQL. Ao longo do texto, o leitor encontrará descrições detalhadas sobre o funcionamento do sistema, incluindo modelos conceitual, lógico e físico, estrutura de procedures, organização da base de dados, estórias de usuários e exemplos práticos de inserção e exclusão lógica de registros. O sistema oferece funcionalidades essenciais como o cadastro de colaboradores e cargos, cálculo de salários com base em bônus e descontos, e exibição dos dados salariais filtrados por cargo, assegurando integridade e confiabilidade nas operações.

Palavras-chave: Banco de Dados; Cálculo de Salários; MySQL; Procedures; View; Exclusão Lógica; Normalização; Cadastro de Funcionários;

1. Introdução

A crescente demanda por sistemas de gerenciamento de informações nas organizações exige soluções que automatizem tarefas operacionais e garantam integridade, eficiência e acessibilidade dos dados. Neste contexto, este trabalho apresenta o desenvolvimento de uma aplicação web, construída com a tecnologia ASP.NET Web Forms, cujo objetivo é gerenciar informações de pessoas e cargos, além de realizar o cálculo automatizado de salários.

A aplicação foi desenvolvida em C# utilizando o banco de dados relacional MySQL para armazenamento persistente dos dados. Sua arquitetura foi organizada em camadas (DAL, BLL e UI), promovendo a separação de responsabilidades e facilitando a manutenção e a escalabilidade do sistema.

Entre as principais funcionalidades do sistema estão: o cadastro e gerenciamento de pessoas, a definição de cargos com salários base, a exclusão lógica de registros (via campo ativo), e a execução automática do cálculo de salários líquidos com base em bônus percentuais aplicados ao salário base. Esse cálculo é realizado através da procedure `calcular_salarios`, que alimenta a tabela `pessoa_salario` e permite a visualização dos resultados por meio da view `vw_pessoa_salario_ativo`.

O sistema também permite a inserção de registros de forma automatizada, por meio da procedure `inserir_pessoa`, garantindo que novos funcionários sejam devidamente relacionados aos seus respectivos cargos. Para facilitar a análise gerencial, o sistema conta com filtros por cargo, paginação de resultados e validações de campos obrigatórios.

O projeto atende aos objetivos estratégicos de uma organização ao oferecer uma solução confiável e automatizada para o controle de dados funcionais e financeiros dos colaboradores. Além disso, a separação entre regras de negócio e interface torna o sistema flexível para adaptações futuras, como ampliação de cálculos com impostos ou integração com folhas de pagamento externas.

2. Modelo Conceitual

2.1 Painei MVP

✓É:

- Um sistema de gerenciamento de cálculo salarial para um corpo de funcionários.
- Uma aplicação web que exibe informações salariais de forma simples e acessível.
- Um sistema baseado em banco de dados com automação de procedimentos.

✗NÃO É:

- Não é um sistema de folha de pagamento completo com integração bancária ou contábil.
- Não possui controle de ponto ou gestão de horas trabalhadas.

✓FAZ:

● Calcula os salários dos funcionários com base em dados de cargo, salário base, impostos e bônus. ● Permite Inserir e excluir funcionários. ● Exibe os dados salariais.
✗ NÃO FAZ: ● Não gera contracheques em PDF. ● Não realiza a transferência bancária dos salários. ● Não permite edição manual dos salários calculados. ● Não gerencia sessão de usuário.

2.1.1 Funcionalidades

Funcionalidade	Descrição
Cadastro de pessoas	Registra dados completos de um colaborador, gerando uma matrícula automática baseada no ano.
Cadastro de cargos	Define os cargos disponíveis na empresa e seus salários base.
Cálculo de salário com bônus	Calcula o salário líquido com base no salário base e um bônus percentual fornecido.
Visualização de salários	Lista os salários líquidos com nome, email e cargo dos funcionários ativos.
Filtro por cargo	Permite visualizar os salários de colaboradores por função.
Exclusão lógica	Permite desativar um funcionário sem perder seus dados.

2.2 Estórias de Usuários

Estória 1 – Cadastro de Funcionário com Matrícula Automática

Como **administrador**, quero cadastrar um novo colaborador e que sua matrícula seja gerada automaticamente

Estória 2 – Cálculo de Salário

Como **gestor**, desejo calcular os salários de todos os colaboradores ativos com base em um bônus percentual e todos os descontos, para gerar os valores líquidos a serem pagos.

Estória 3 – Visualização de Salários Ativos

Como **gestor financeiro**, quero visualizar os salários líquidos dos colaboradores ativos, com seus respectivos nomes, e-mails e cargos.

3. Modelo Lógico

O modelo de dados desenvolvido encontra-se na **Terceira Forma Normal (3FN)**, que assegura:

1FN:

Todas as tabelas possuem atributos **atômicos e únicos por campo**, sem repetições ou listas múltiplas.

Exemplo: o endereço está devidamente dividido em campos como logradouro, numero, cidade, cep, etc.

Satisfaz 1FN

2FN:

Todas as tabelas estão da 1FN.

Todos os atributos não-chave dependem da chave primária, não há dependência parcial de chaves compostas (não há chave composta).

Satisfaz 2FN

3FN:

Garantida a 2FN.

Não há dependências transitivas entre atributos não-chave.

Exemplo: o salário base é armazenado em cargo, e não diretamente em pessoa, evitando redundância.

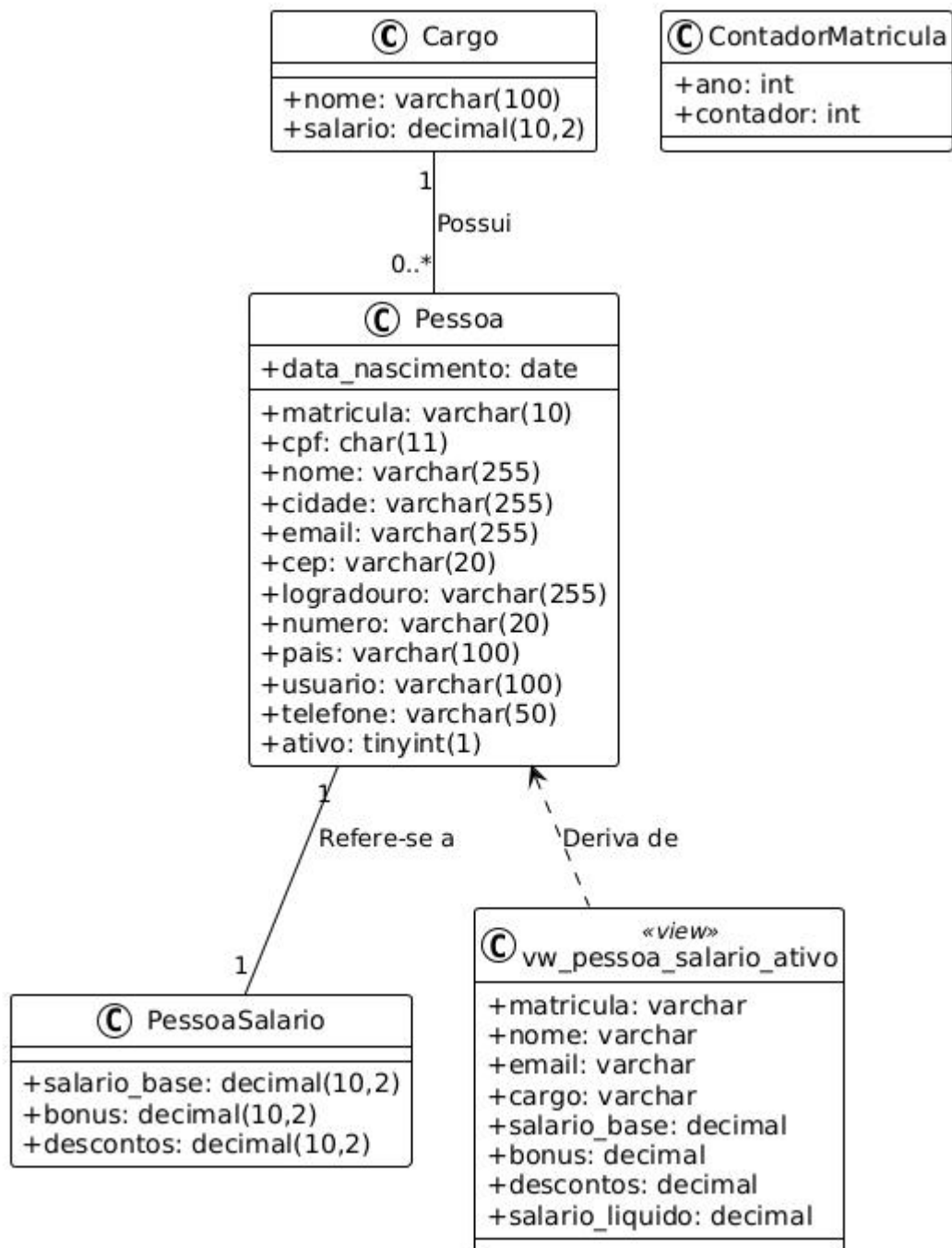
Exemplo: o salário líquido é calculado de forma dinâmica diretamente na view, não sendo um dado físico, evitando dependência transitiva nos cálculos de salário.

Satisfaz 3FN

Essa estrutura garante consistência, elimina redundância e facilita futuras manutenções e expansões no sistema.

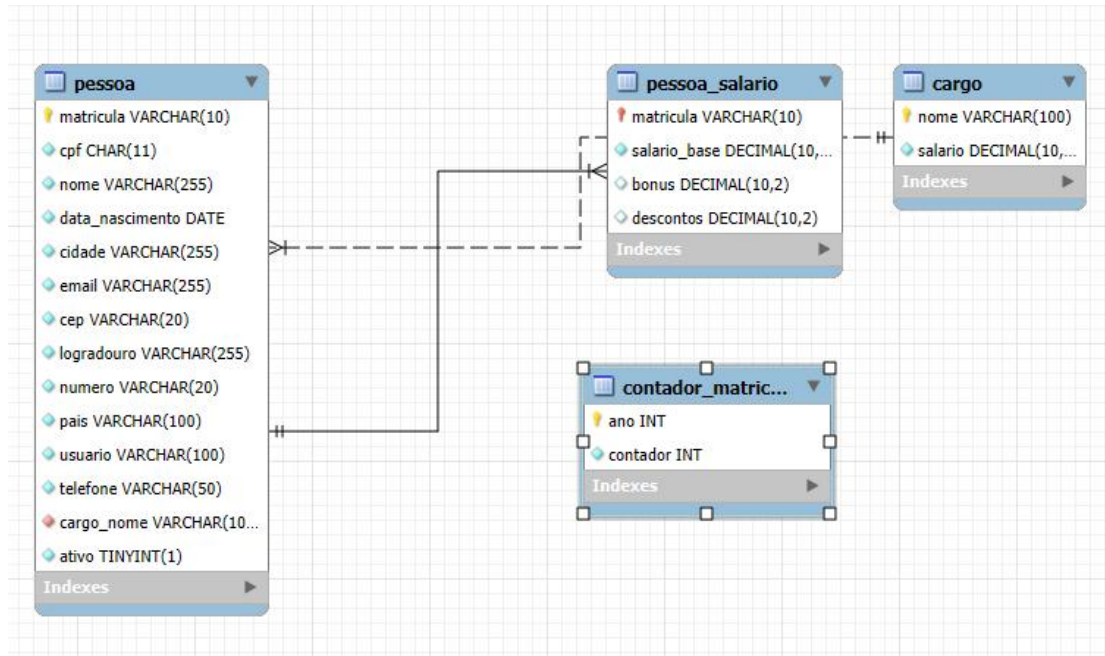
3.1 Diagrama Entidade Relacionamento

Modelo lógico representativo das entidades, gerado em **PlantUML**.



4. Modelo Físico

O diagrama a seguir contém as entidades relacionadas; pessoa; cargo; pessoa_salario e uma entidade especial contador_matrícula, usada para gerenciamento da criação da matrícula da entidade pessoa, na procedure inserir_pessoa.



O modelo físico final ficou da seguinte forma, para cada entidade, e é possível identificar suas chaves primárias e relacionamentos:

Pessoa:

```
1 CREATE TABLE `pessoa` (  
2   `matricula` varchar(10) NOT NULL,  
3   `cpf` char(11) NOT NULL,  
4   `nome` varchar(255) NOT NULL,  
5   `data_nascimento` date NOT NULL,  
6   `cidade` varchar(255) NOT NULL,  
7   `email` varchar(255) NOT NULL,  
8   `cep` varchar(20) NOT NULL,  
9   `logradouro` varchar(255) NOT NULL,  
10  `numero` varchar(20) NOT NULL,  
11  `pais` varchar(100) NOT NULL,  
12  `usuario` varchar(100) NOT NULL,  
13  `telefone` varchar(50) NOT NULL,  
14  `cargo_nome` varchar(100) NOT NULL,  
15  `ativo` tinyint(1) NOT NULL DEFAULT '1',  
16  PRIMARY KEY (`matricula`),  
17  UNIQUE KEY `email` (`email`),  
18  UNIQUE KEY `cpf_UNIQUE` (`cpf`),  
19  UNIQUE KEY `usuario_UNIQUE` (`usuario`),  
20  KEY `cargo_nome` (`cargo_nome`),  
21  CONSTRAINT `pessoa_ibfk_1` FOREIGN KEY (`cargo_nome`) REFERENCES `cargo` (`nome`)  
22 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Cargo:

```
1 CREATE TABLE `cargo` (  
2   `nome` varchar(100) NOT NULL,  
3   `salario` decimal(10,2) NOT NULL,  
4   PRIMARY KEY (`nome`)  
5 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Pessoa-salario:

```
1 CREATE TABLE `pessoa_salario` (  
2   `matricula` varchar(10) NOT NULL,  
3   `salario_base` decimal(10,2) NOT NULL,  
4   `bonus` decimal(10,2) DEFAULT '0.00',  
5   `descontos` decimal(10,2) DEFAULT '0.00',  
6   PRIMARY KEY (`matricula`),  
7   CONSTRAINT `pessoa_salario_ibfk_1` FOREIGN KEY (`matricula`) REFERENCES `pessoa` (`matricula`)  
8 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Contador_matricula:

```
1 CREATE TABLE `contador_matricula` (  
2   `ano` int NOT NULL,  
3   `contador` int NOT NULL,  
4   PRIMARY KEY (`ano`)  
5 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

4.1 Como é feito a consulta

A consulta dos salários é feita através de uma view `vw_pessoa_salario_ativo`, que consulta todos os funcionários ativos no sistema, e faz o calculo do salário líquido dinamicamente a cada consulta.

4.1.1 Estrutura da view

```
CREATE
  ALGORITHM = UNDEFINED
  DEFINER = `root`@`localhost`
  SQL SECURITY DEFINER
VIEW `vw_pessoa_salario_ativo` AS
  SELECT
    `p`.`matricula` AS `matricula`,
    `p`.`nome` AS `nome`,
    `p`.`email` AS `email`,
    `p`.`cargo_nome` AS `cargo`,
    `ps`.`salario_base` AS `salario_base`,
    `ps`.`bonus` AS `bonus`,
    `ps`.`descontos` AS `descontos`,
    ((`ps`.`salario_base` + `ps`.`bonus`) - `ps`.`descontos`) AS `salario_liquido`
  FROM
    (`pessoa` `p`
    JOIN `pessoa_salario` `ps` ON ((`p`.`matricula` = `ps`.`matricula`)))
  WHERE
    (`p`.`ativo` = 1)
```

4.2 Manipulação

O projeto possui duas procedures responsáveis pela manipulação dos dados, se encarregando de calcular e popular a tabela `pessoa_salario`, com a procedure **calcular_salarios**, e o gerenciamento dos funcionários, sendo possível adicionar novos funcionários e seu respectivo cargo, com a procedure **inserir_pessoa**.

4.2.1 Estrutura calcular_salarios

Essa procedure recebe o bônus e o desconto que serão usados no calculo final, e consulta o valor do salário base para cada cargo da consulta.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `calcular_salarios` (
    IN p_bonus DECIMAL(10,2),
    IN p_descontos DECIMAL(10,2)
)
BEGIN
    DECLARE v_bonus DECIMAL(10,2);
    DECLARE v_descontos DECIMAL(10,2);

    SET v_bonus = IFNULL(p_bonus, 0);
    SET v_descontos = IFNULL(p_descontos, 0);

    TRUNCATE TABLE pessoa_salario;

    INSERT INTO pessoa_salario (matricula, salario_base, bonus, descontos)
    SELECT
        p.matricula,
        c.salario AS salario_base,
        c.salario * v_bonus / 100 AS bonus,
        v_descontos AS descontos
    FROM pessoa p
    INNER JOIN cargo c ON p.cargo_nome = c.nome
    WHERE p.ativo = 1;
END
```

4.2.2 Estrutura inserir_pessoa

A procedure mais importante, que trata de inserir dados permanente no banco, obedecendo a estrutura da tabela, garantindo a persistência dos dados.

Ela também se encarrega de gerenciar a criação da chave primária de **pessoa**, manipulando o contador para a criação da matrícula para cada ano, garantindo unicidade da chave primária.

```

CREATE DEFINER=`root`@`localhost` PROCEDURE `inserir_pessoa`(
  IN p_nome VARCHAR(100),
  IN p_cidade VARCHAR(100),
  IN p_email VARCHAR(100),
  IN p_cep VARCHAR(20),
  IN p_logradouro VARCHAR(255),
  IN p_numero VARCHAR(20),
  IN p_pais VARCHAR(50),
  IN p_usuario VARCHAR(50),
  IN p_telefone VARCHAR(30),
  IN p_data_nascimento DATE,
  IN p_cargo_nome VARCHAR(100),
  IN p_cpf CHAR(11)
)
BEGIN
  DECLARE ano_atual INT;
  DECLARE contador_atual INT;
  DECLARE nova_matricula VARCHAR(10);

  SET ano_atual = YEAR(CURDATE());

  -- Se ainda não há entrada para o ano atual, criar
  INSERT INTO contador_matricula (ano, contador)
  VALUES (ano_atual, 0)
  ON DUPLICATE KEY UPDATE contador = contador;

  -- Incrementar o contador
  UPDATE contador_matricula
  SET contador = contador + 1
  WHERE ano = ano_atual;

  -- Obter valor atualizado
  SELECT contador INTO contador_atual
  FROM contador_matricula
  WHERE ano = ano_atual;

  -- Gerar a matrícula no formato YYNNN
  SET nova_matricula = CONCAT(LPAD(RIGHT(ano_atual, 2), 2, '0'), LPAD(contador_atual, 2, '0'));

  -- Inserir a pessoa com a matrícula gerada
  INSERT INTO pessoa (
    matricula, nome, cidade, email, cep, logradouro, numero, pais,
    usuario, telefone, data_nascimento, cargo_nome, cpf, ativo
  ) VALUES (
    nova_matricula, p_nome, p_cidade, p_email, p_cep, p_logradouro, p_numero, p_pais,
    p_usuario, p_telefone, p_data_nascimento, p_cargo_nome, p_cpf, 1
  );
END

```

4.3 Exemplos de Inserção

O Banco pode ser populado de forma manual ou pela aplicação, de preferência pela aplicação, onde não há a necessidade de um profissional em banco de dados para a manipulação.

4.3.1 Como exemplo de inserção manual

```
INSERT INTO pessoa (matricula, cpf, nome, data_nascimento, cidade, email, cep,
logradouro, numero, pais, usuario, telefone, cargo_nome, ativo) VALUES
(2511, '12345678900', 'Ana Clara', '1995-03-10', 'Natal', 'ana@gmail.com', '59000000',
'Rua das Flores', '100', 'Brasil', 'ana_user', '(84) 98888-1111', 'Analista', 1)
```

O comando SQL em questão insere na tabela **pessoa** as informações pertinentes ao funcionário, mas é necessário atenção para garantir que o número de matrícula e o cargo esteja em conformes com a estrutura vigente.

4.3.2 Exemplo de inserção automática

Via aplicação, a procedure **inserir_pessoa** se garante de manter toda a persistência e normalização.

Adicionar Nova Pessoa

Salarios

Nome:

Cidade:

Email:

CEP:

Logradouro:

Numero:

País:

Usuário:

Telefone:

Data de Nascimento:

Cargo:

 ▼

CPF:

Adicionar

Essa abordagem mantém inserção segura e menos suscetível a erros de operação. Essa tela preenche os campos da procedure que foram mostrados no item 4.2.2.

5. Interfaces

A aplicação possui duas interfaces principais. A primeira é responsável por listar os funcionários, com matrícula, nome, email, cargo, bonus, descontos e salario_liquido, conforme a estrutura da view, no item 4.1.1.



Salários

[Ir para cadastro de Pessoas](#)

Todos ▼ Bônus: 20 Descontos: 159 [Calcular/Recalcular](#)

Matricula	Nome	Email	Cargo	SalarioBase	Bonus	Descontos	SalarioLiquido
2503	Ana Louro	ana@yahoo.com	Estagiario	1500.00	300.00	159.00	1641.00
2504	João Da Silva	joao@gmail.com	Gerente	9000.00	1800.00	159.00	10641.00
2505	Joao Fontes	ffo@gmail.com	Coordenador	6000.00	1200.00	159.00	7041.00
2506	Severina Pato	svr@gmail.com	Gerente	9000.00	1800.00	159.00	10641.00
2507	Antonio Pereira	ap@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2509	Melkor Primeiro	mlkor@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2511	Ana Clara	ana@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2512	Bruno Silva	bruno@gmail.com	Técnico	2500.00	500.00	159.00	2841.00
2513	Carla Souza	carla@gmail.com	Gerente	9000.00	1800.00	159.00	10641.00
2514	Diego Lima	diego@gmail.com	Coordenador	6000.00	1200.00	159.00	7041.00
2515	Eduarda Matos	eduarda@gmail.com	Estagiário	1500.00	300.00	159.00	1641.00
2516	Felipe Torres	felipe@gmail.com	Técnico	2500.00	500.00	159.00	2841.00
2517	Gabriela Rocha	gabriela@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2518	Henrique Dantas	henrique@gmail.com	Técnico	2500.00	500.00	159.00	2841.00
2519	Isabela Melo	isabela@gmail.com	Gerente	9000.00	1800.00	159.00	10641.00
2520	João Pedro	joa67o@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2521	Karen Alves	karen@gmail.com	Estagiário	1500.00	300.00	159.00	1641.00
2522	Lucas Freitas	lucas@gmail.com	Técnico	2500.00	500.00	159.00	2841.00
2523	Marina Cunha	marina@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2524	Nicolas Dias	nicolas@gmail.com	Coordenador	6000.00	1200.00	159.00	7041.00

< << 1 2

Página 1 de 2

[⬅ Voltar Página](#) [Próxima Página ➡](#)

A segunda, como mostrado no item 4.3.2. É responsável por inserir um novo colaborador ao banco de dados da empresa, garantindo os pontos abordados. Ao final da operação, é emitido a mensagem de sucesso, ou erro (caso os dados não estejam conforme esperado).

Abaixo segue uma demonstração de inserção de funcionário.

Salarios

Nome:

Cidade:

Email:

CEP:

Logradouro:

Numero:

País:

Usuário:

Telefone:

Data de Nascimento:

Cargo:

CPF:

Nome:

Cidade:

Email:

CEP:

Logradouro:

Numero:

País:

Usuário:

Telefone:

Data de Nascimento:

Cargo:

CPF:

Pessoa adicionada com sucesso!

A segunda interface também trata da exclusão de funcionários, mas a aplicação trata de manter o histórico no banco, realizando apenas a exclusão lógica, desabilitando o funcionário dos cálculos da folha.

Segue demonstração de uma exclusão:

Salários

[Ir para cadastro de Pessoas](#)

Todos ▼ Bônus: Descontos:

Matricula	Nome	Email	Cargo	SalarioBase	Bonus	Descontos	SalarioLiquido
2503	Ana Louro	ana@yahoo.com	Estagiario	1500.00	300.00	159.00	1641.00
2504	João Da Silva	joao@gmail.com	Gerente	9000.00	1800.00	159.00	10641.00
2505	Joao Fontes	ffo@gmail.com	Coordenador	6000.00	1200.00	159.00	7041.00
2506	Severina Pato	svr@gmail.com	Gerente	9000.00	1800.00	159.00	10641.00
2507	Antonio Pereira	ap@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2509	Melkor Primeiro	mlkor@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2511	Ana Clara	ana@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2512	Bruno Silva	bruno@gmail.com	Técnico	2500.00	500.00	159.00	2841.00
2513	Carla Souza	carla@gmail.com	Gerente	9000.00	1800.00	159.00	10641.00
2514	Diego Lima	diego@gmail.com	Coordenador	6000.00	1200.00	159.00	7041.00
2515	Eduarda Matos	eduarda@gmail.com	Estagiário	1500.00	300.00	159.00	1641.00
2516	Felipe Torres	felipe@gmail.com	Técnico	2500.00	500.00	159.00	2841.00
2517	Gabriela Rocha	gabriela@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2518	Henrique Dantas	henrique@gmail.com	Técnico	2500.00	500.00	159.00	2841.00
2519	Isabela Melo	isabela@gmail.com	Gerente	9000.00	1800.00	159.00	10641.00
2520	João Pedro	joa67o@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2521	Karen Alves	karen@gmail.com	Estagiário	1500.00	300.00	159.00	1641.00
2522	Lucas Freitas	lucas@gmail.com	Técnico	2500.00	500.00	159.00	2841.00
2523	Marina Cunha	marina@gmail.com	Analista	4000.00	800.00	159.00	4641.00
2524	Nicolas Dias	nicolas@gmail.com	Coordenador	6000.00	1200.00	159.00	7041.00

<< 1 2

Página 1 de 2

Excluindo:

Excluir Pessoa

Matricula da Pessoa: Pessoa excluída com sucesso.

Verificando o sucesso da operação:

Salários

[Ir para cadastro de Pessoas](#)

Todos ▼ Bônus: Descontos:

Matricula	Nome	Email	Cargo	SalarioBase	Bonus	Descontos	SalarioLiquido
2504	João Da Silva	joao@gmail.com	Gerente	9000.00	0.00	0.00	9000.00
2505	Joao Fontes	ffo@gmail.com	Coordenador	6000.00	0.00	0.00	6000.00
2506	Severina Pato	svr@gmail.com	Gerente	9000.00	0.00	0.00	9000.00
2507	Antonio Pereira	ap@gmail.com	Analista	4000.00	0.00	0.00	4000.00
2509	Melkor Primeiro	mlkor@gmail.com	Analista	4000.00	0.00	0.00	4000.00
2511	Ana Clara	ana@gmail.com	Analista	4000.00	0.00	0.00	4000.00
2512	Bruno Silva	bruno@gmail.com	Técnico	2500.00	0.00	0.00	2500.00
2513	Carla Souza	carla@gmail.com	Gerente	9000.00	0.00	0.00	9000.00
2514	Diego Lima	diego@gmail.com	Coordenador	6000.00	0.00	0.00	6000.00
2515	Eduarda Matos	eduarda@gmail.com	Estagiário	1500.00	0.00	0.00	1500.00
2516	Felipe Torres	felipe@gmail.com	Técnico	2500.00	0.00	0.00	2500.00
2517	Gabriela Rocha	gabriela@gmail.com	Analista	4000.00	0.00	0.00	4000.00
2518	Henrique Dantas	henrique@gmail.com	Técnico	2500.00	0.00	0.00	2500.00
2519	Isabela Melo	isabela@gmail.com	Gerente	9000.00	0.00	0.00	9000.00
2520	João Pedro	joa67o@gmail.com	Analista	4000.00	0.00	0.00	4000.00
2521	Karen Alves	karen@gmail.com	Estagiário	1500.00	0.00	0.00	1500.00
2522	Lucas Freitas	lucas@gmail.com	Técnico	2500.00	0.00	0.00	2500.00
2523	Marina Cunha	marina@gmail.com	Analista	4000.00	0.00	0.00	4000.00
2524	Nicolas Dias	nicolas@gmail.com	Coordenador	6000.00	0.00	0.00	6000.00
2525	Olivia Batista	olivia@gmail.com	Gerente	9000.00	0.00	0.00	9000.00

< << 1 2

Página 1 de 2

Nota-se que a matrícula 2503 não está mais presente na consulta, já que o Banco elimina todos os campos inativos de suas operações.

6. Conclusão

Este projeto teve como objetivo o desenvolvimento de uma aplicação focada em banco de dados, para a disciplina Banco De Dados - DCO3035, da UFRN. O intuito final foi adquirir a habilidade de desenvolvimento de um banco de dados robusto e normalizado, que garanta a persistência de dados e a integridade da informação.

Como discente, acredito que o objetivo fora alcançado e a aplicação representa um sistema de folha de pagamento para pequenas empresas, garantindo automação e integridade em operações que muitas vezes ficam a cargos de planilhas e trabalho manual.